Glaz Documentation: Automated Penetration Testing Tool

By: Denis Albin
Email: albindeniswork@gmail.com

This project provides an efficient framework for automated network penetration testing and vulnerabilities detection. The script Glaz begins by verifying its environment and ensuring all necessary tools are present before guiding the user through a detailed, interactive scan configuration. Its primary function is to perform network reconnaissance, map discovered services to known vulnerabilities, and audit common login services for weak credentials. All findings are logged and organized for clear, actionable reporting.

Key capabilities of the Glaz include:

- Automated verification and installation of necessary penetration testing tools (Nmap, Hydra, Searchsploit).
- Interactive menu for configuring scan parameters, including scan type, aggressiveness, and target ports.
- Dual-mode operation: a fast 'Basic' scan for service discovery and a 'Full' scan that adds deep vulnerability analysis with the Nmap Scripting Engine (NSE) and Searchsploit.
- Systematic auditing of weak credentials for services like SSH, FTP, SMB, and Telnet using Hydra and CrackMapExec.
- Structured reporting with results saved in multiple formats (log, XML, HTML) within timestamped directories.
- Optional automatic compression of all results into a single Zip archive for portability.

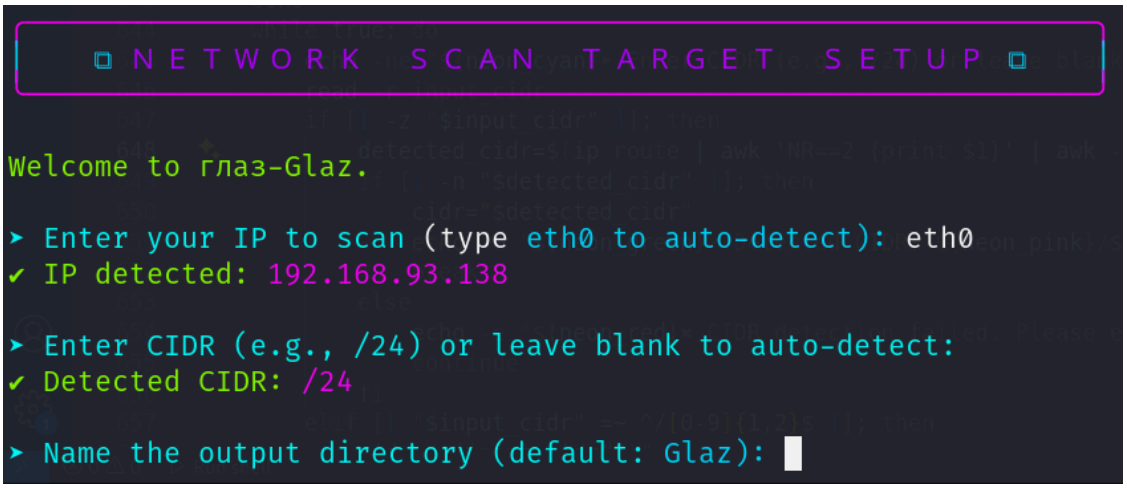
1. Getting the User Input

1.1 Get from the user a network to scan. && 1.2 Get from the user a name for the output directory.

The **ip_input** function provides a user-friendly interface to define the scope of the test. It prompts for a target IP address and CIDR mask. The function includes input validation to prevent errors and offers an auto-detection feature for both IP and CIDR by reading the configuration of the eth0 interface.

It also lets the user to define the name of the output directory (the default is Glaz)

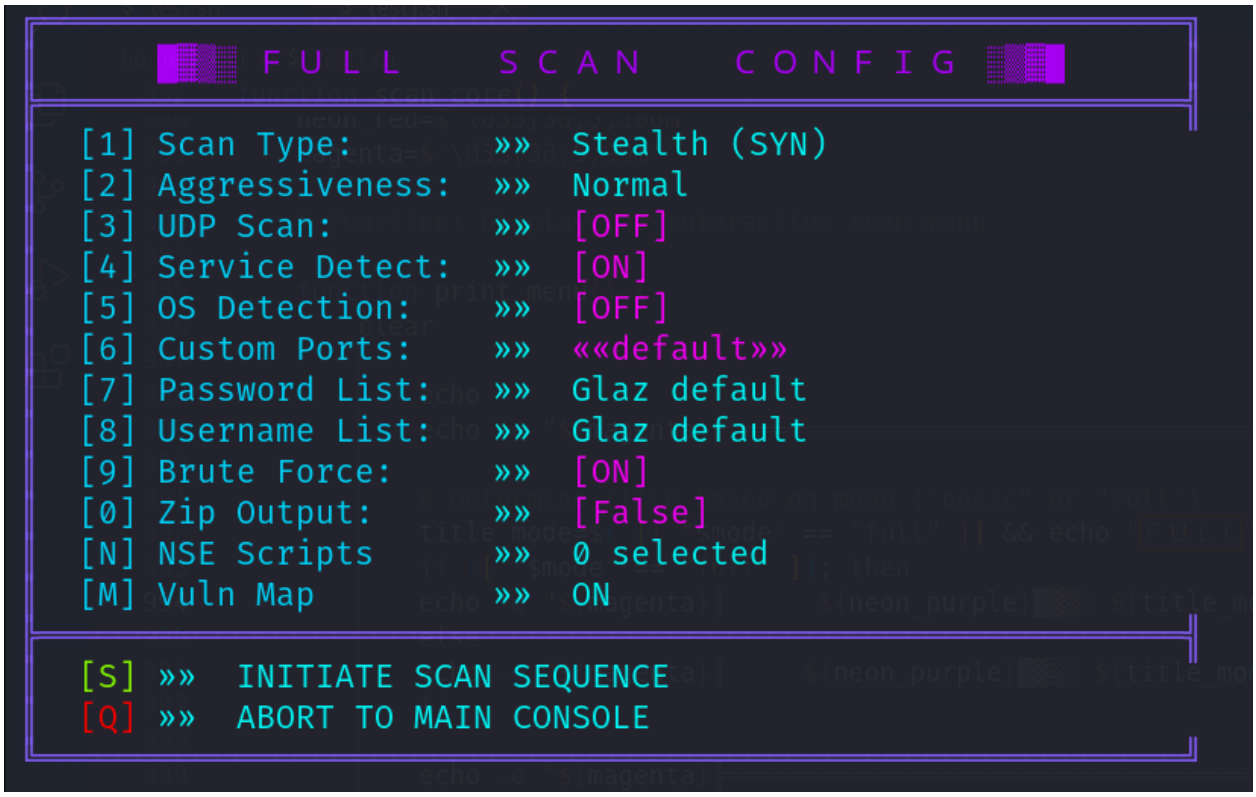
Example output for target acquisition:



1.3 Scan Mode Selection – 1.3.1 Basic (TCP/UDP, Service Version, Weak Passwords) | 1.3.2 Full (NSE, Weak Passwords, Vulnerability Analysis)

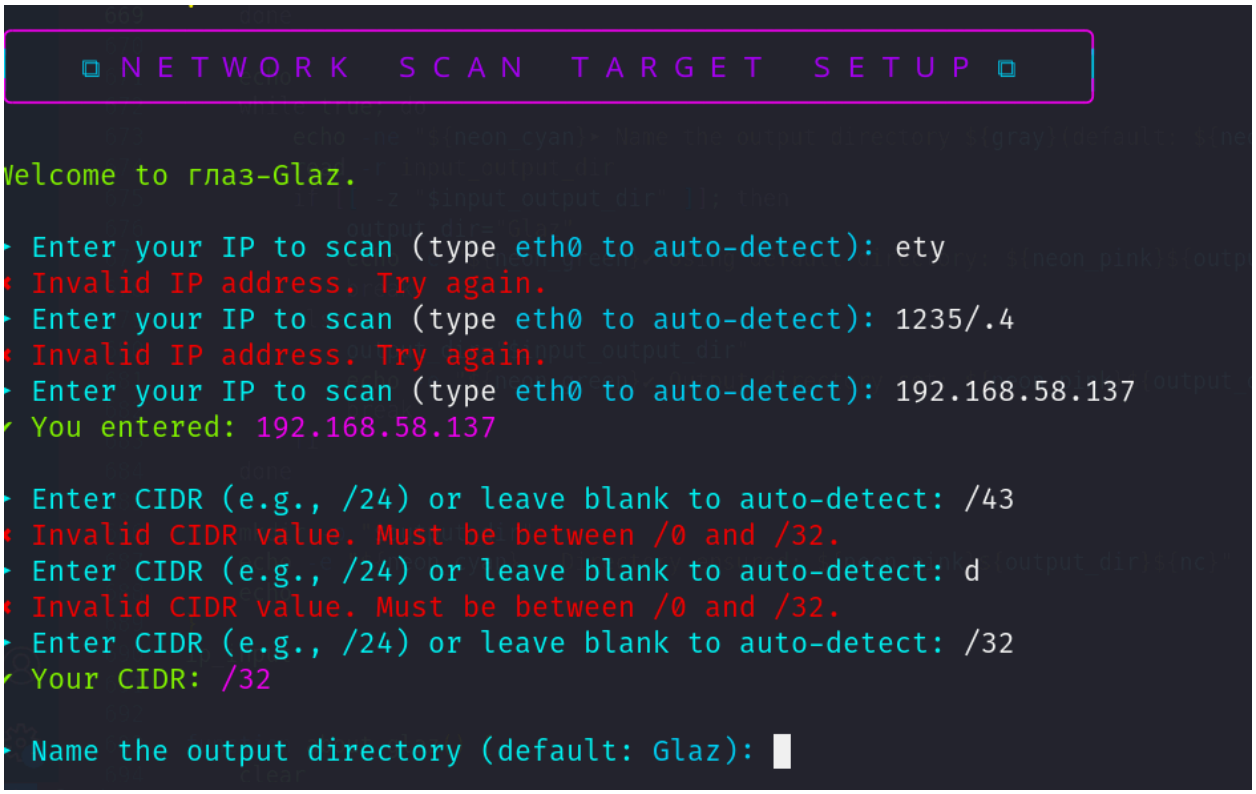
The **scan_core** function presents a detailed, interactive menu where the user can configure the scan. The user can select between a **Basic Scan** (TCP/UDP discovery and weak password checks) and a **Full Scan** (adds NSE scripts and vulnerability mapping). Further options allow for toggling UDP scans, OS/Service detection, specifying custom ports, and selecting custom wordlists for brute-force attacks or leaving it as default for Glaz's built in password list.

Example output of the configuration menu:



1.4 Make sure the input is valid.

Throughout the script there are many error handling functionalities for example:

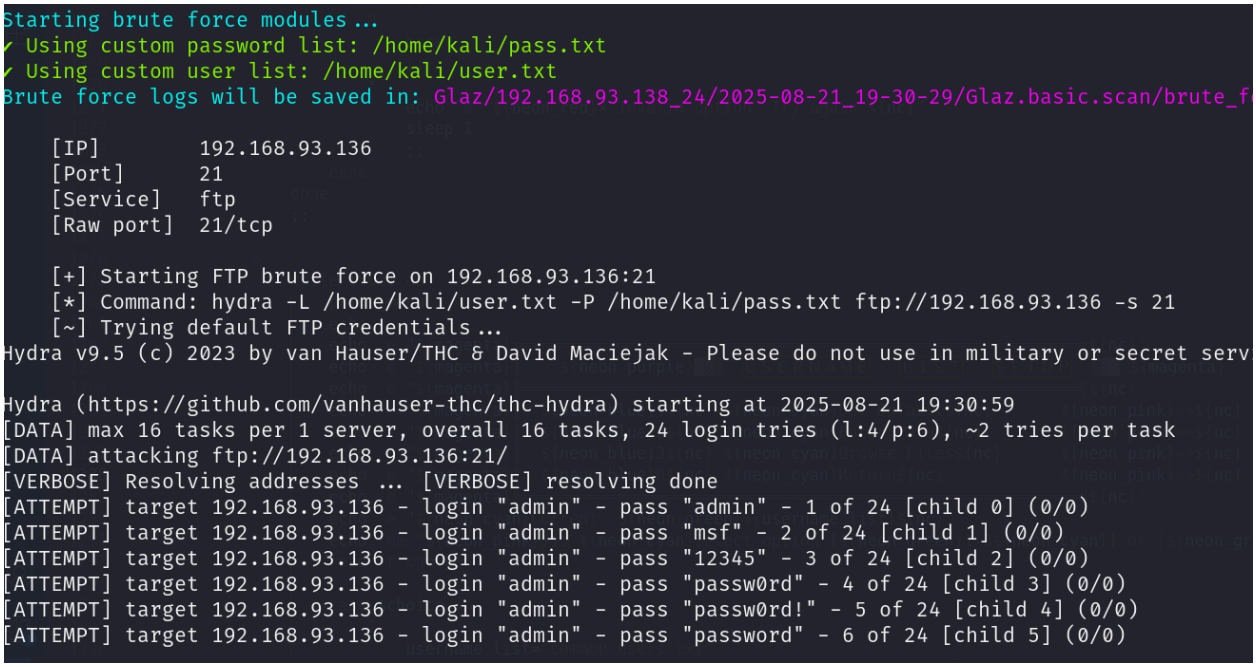


2. Weak Credentials

2.1 Look for weak passwords used in the network for login services. && 2.2 Login services to check include: SSH, RDP, FTP, and TELNET.

If the "Brute Force" option is enabled, the script parses the Nmap scan results to identify hosts running common login services. For each identified service, it launches a targeted attack using Hydra (for SSH, FTP, Telnet, SMTP etc..) to test for weak passwords using either a built-in default list or a user-provided wordlist.

Example output for the brute-force stage:



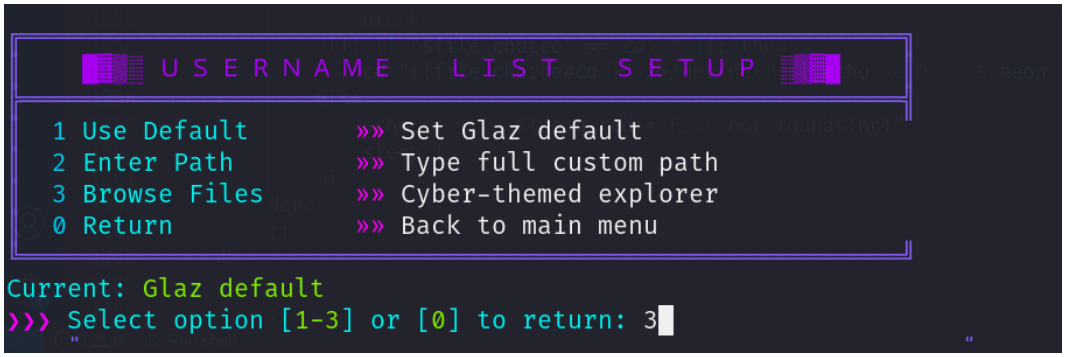


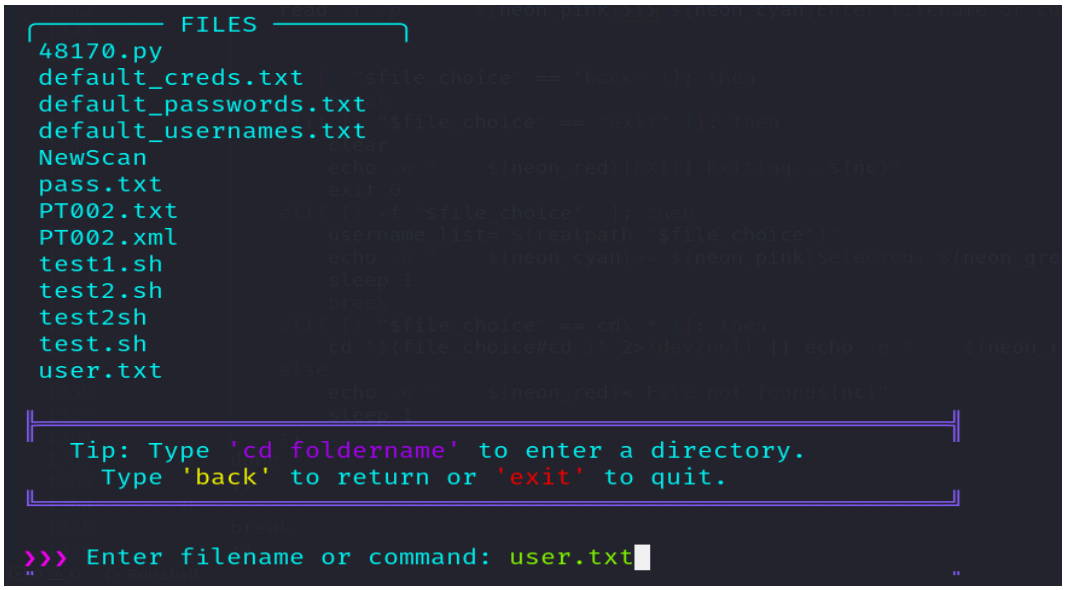
2.1.1 Have a built-in password.lst to check for weak passwords. && 2.1.2 Allow the user to supply their own password list.

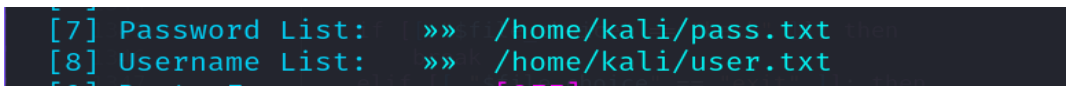
Glaz offers the user to either use a custom password/user list or use its own default list which is set by default
As for the current version it contains many different passwords and users that add up to 30132 login attempts.

```
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-08-21 20:22:38
[DATA] max 16 tasks per 1 server, overall 16 tasks, 30132 login tries (l:124/p:243), ~1884 tries per task
[DATA] attacking ftp://192.168.93.136:21/
```

And furthermore Glaz lets the user to choose their own password/user list







3. Mapping Vulnerabilities

3.1 Mapping vulnerabilities should only take place if Full was chosen. && 3.2 Display potential vulnerabilities via NSE and Searchsploit.

When Full Scan is selected and the Vuln Map option is enabled, if the user disables the Service Detection configuration, the Vuln Map option is automatically disabled as well.

Likewise, if the user attempts to enable the Vuln Map configuration while Service Detection is disabled, the script will automatically enable Service Detection.

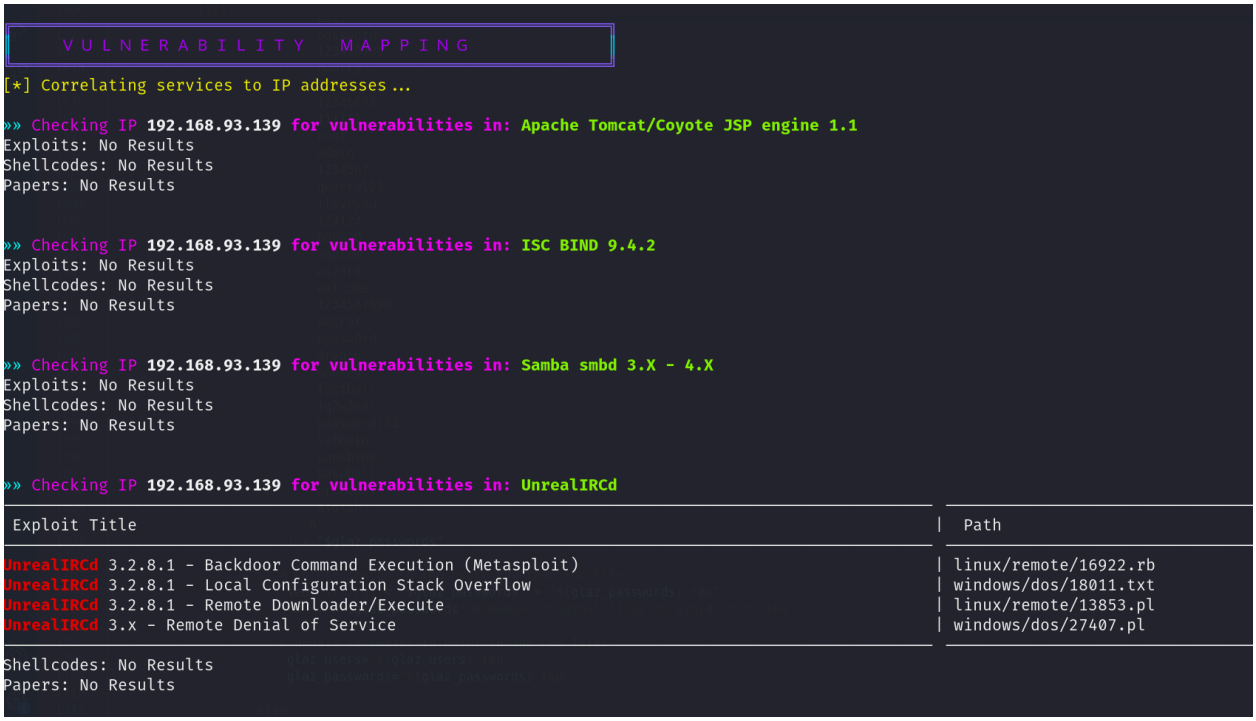




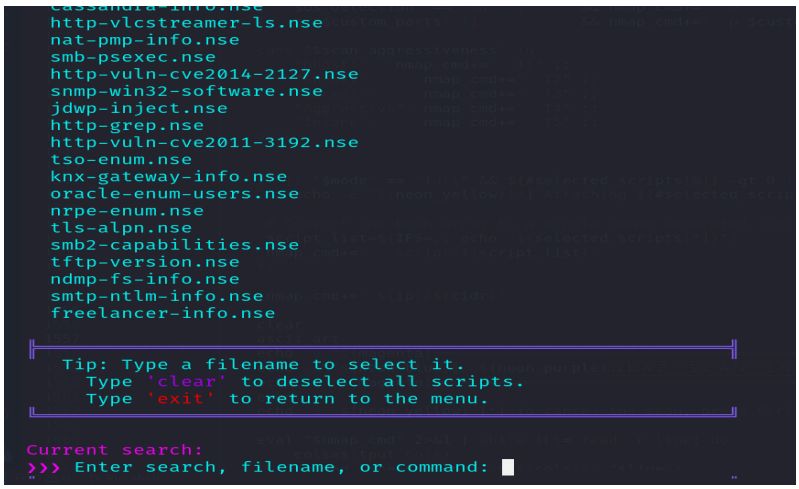
The script automates the process of mapping discovered services to known vulnerabilities. It first uses xmlstarlet to parse the Nmap XML output, extracting a list of each service's product and version, correctly correlated with its host IP address and then uses Searchsploit to find any available exploits on each service

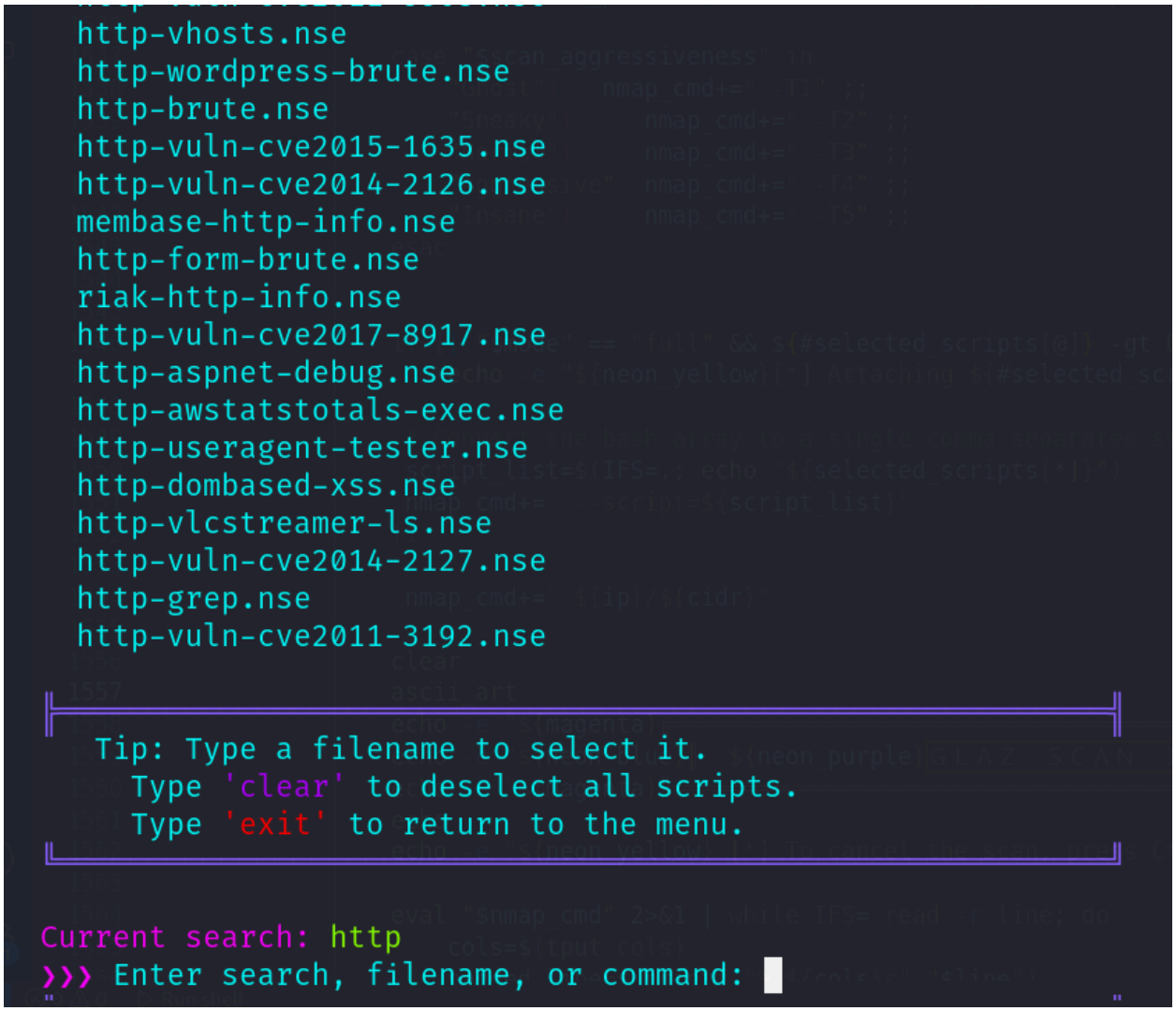
Before each time searchsploit is being used to find exploitable services it is being updated each time via “Searchsploit -u”

Example output for the vulnerability mapping stage:



For NSE scripting, the Full Scan has a built-in browser that displays Nmap scripts. It includes search capabilities to help the user find specific scripts more easily.









Scan Summary

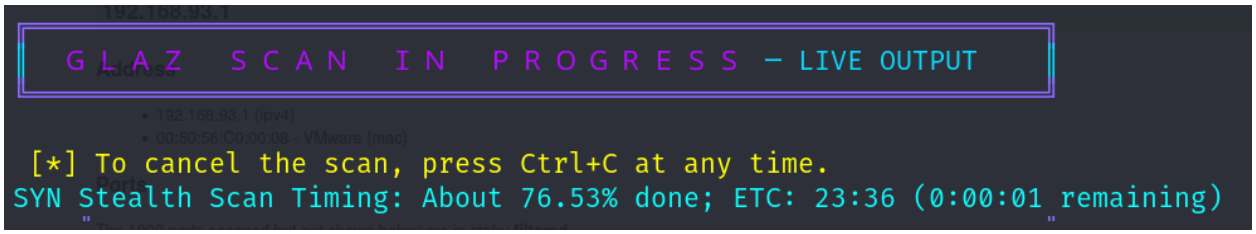
Nmap 7.94SVN was initiated at Thu Aug 21 22:26:49 2025 with these arguments:
/usr/lib/nmap/nmap --stats-every 0.5s -v -oX Glaz/192.168.93.138_24/2025-08-21_22-26-49/Glaz.full.scan/Xml/scan.xml -oN Glaz/192.168.93.138_24/2025-08-21_22-26-49/Glaz.full.scan/scan.log -sS -sV -T3 --script=http-grep.nse,smb2-vuln-uptime.nse 192.168.93.138/24

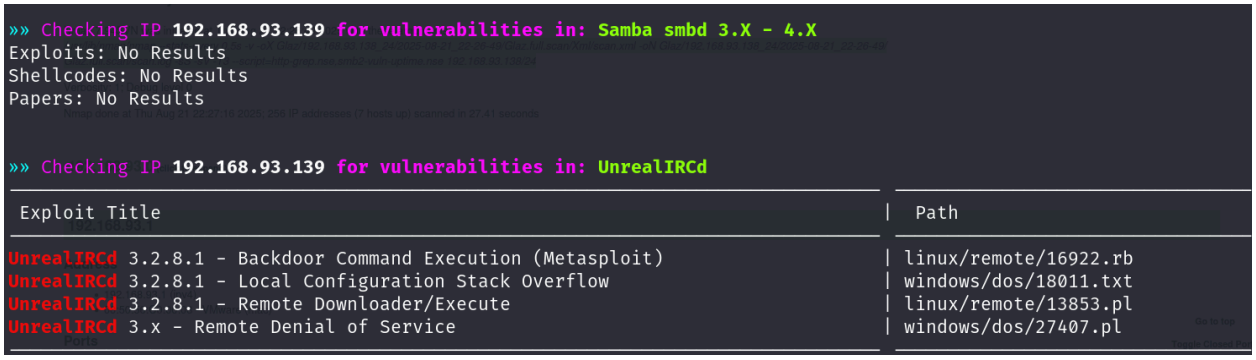
4. Log Results

4.1 During each stage, display the stage in the terminal.

Throughout each stage of its operation, Glaz provides a live feed of its actions and findings. As it runs commands like Nmap for port scanning or Searchsploit for vulnerability mapping, all the output is shown directly to the user in real time, ensuring they can monitor the entire process as it unfolds.

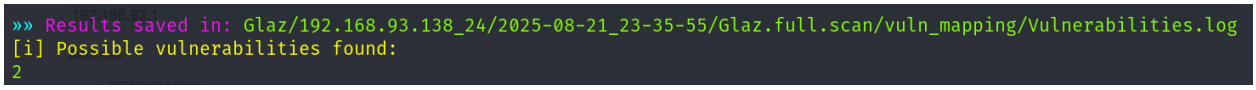
Some examples:

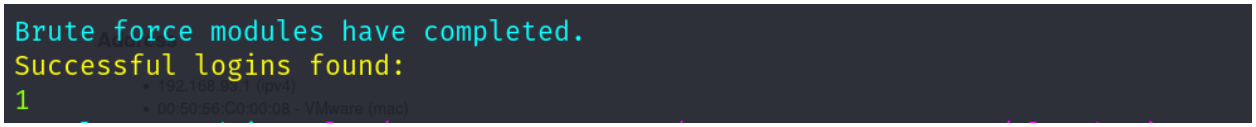




4.2 At the end, show the user the found information.

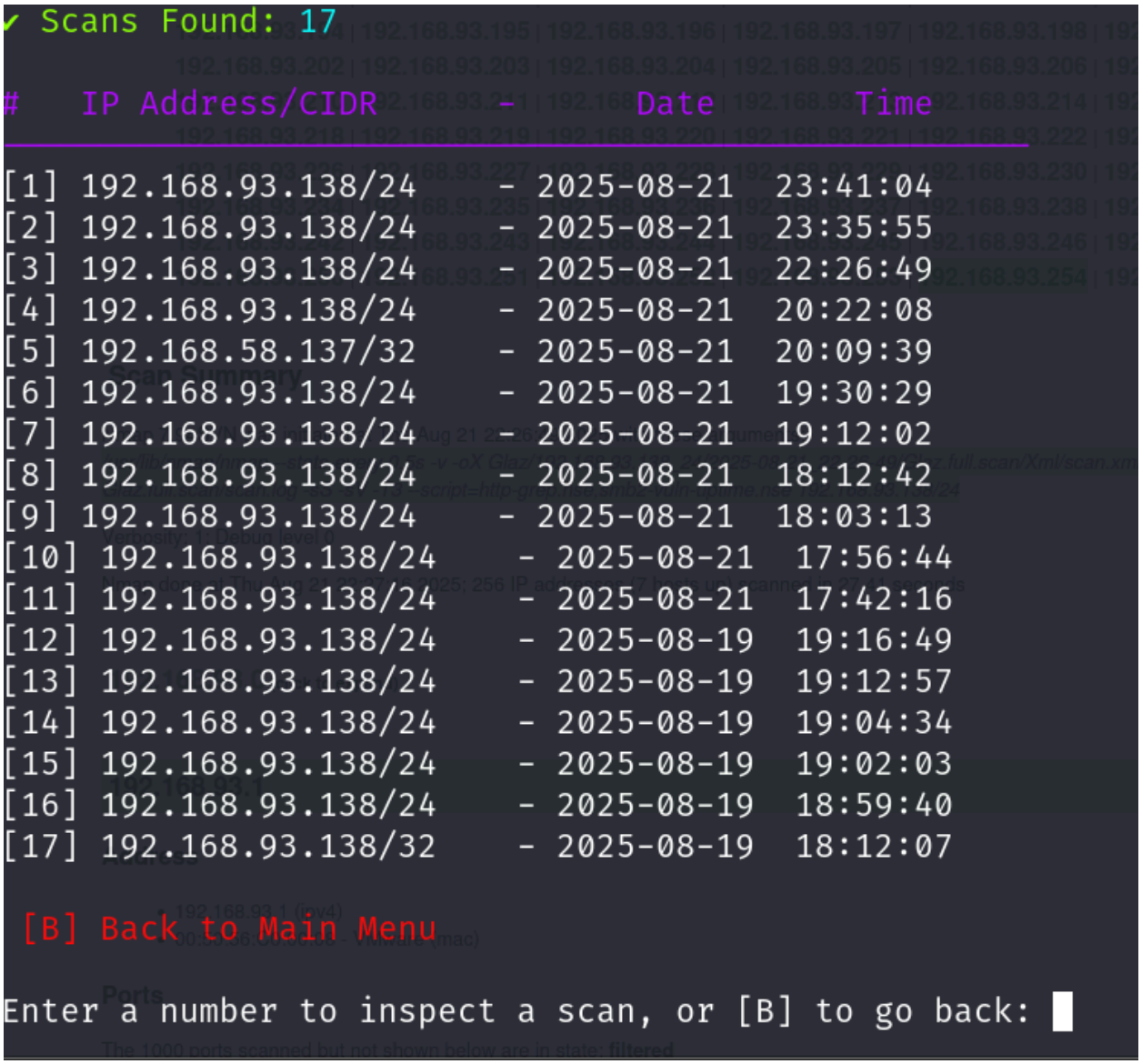
Upon completion of each stage, Glaz shows a summary of all the critical information it has gathered. For instance,for example once the vulnerability mapping is finished, it provides a final count of potential exploits found, ensuring the user has a clear and immediate understanding of the results before the script proceeds to the next phase.

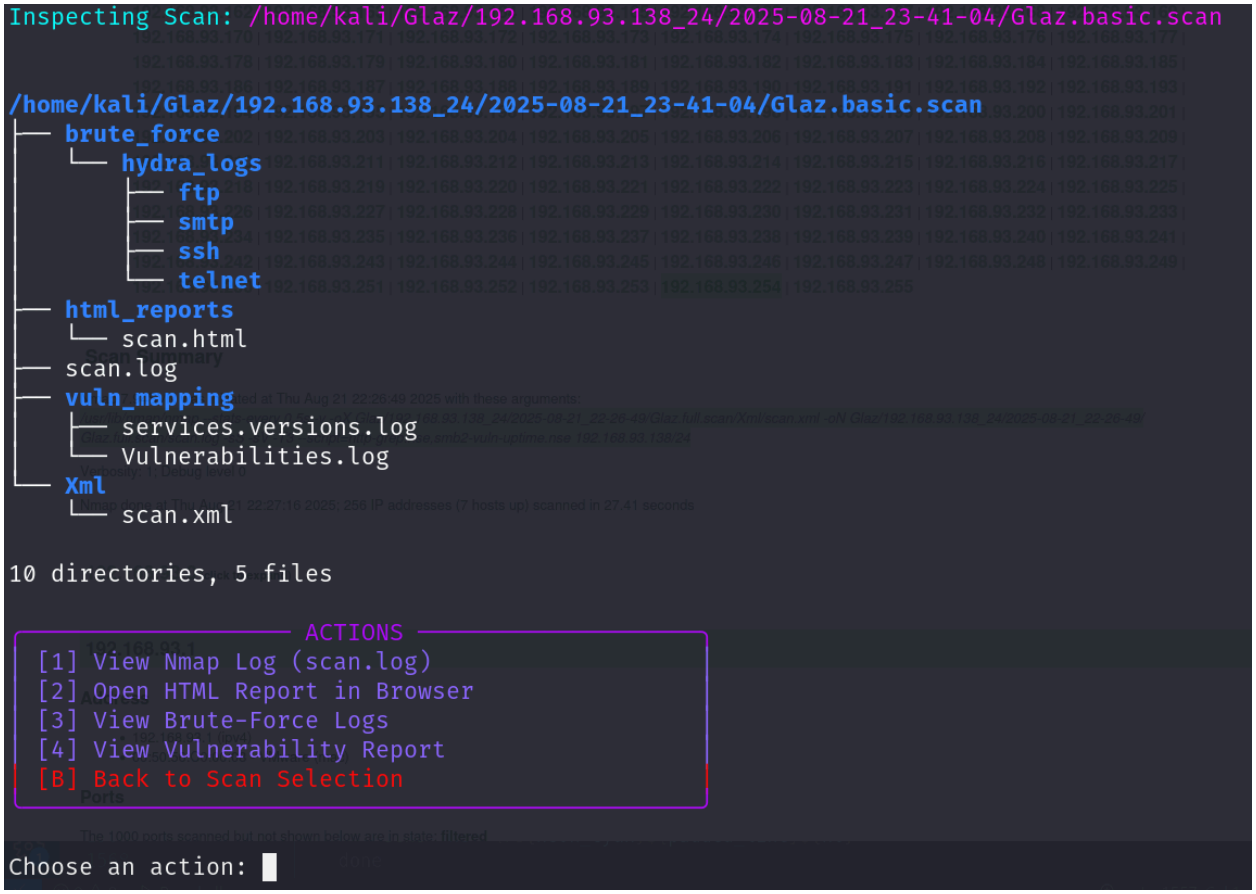




4.3 Allow the user to search inside the results.

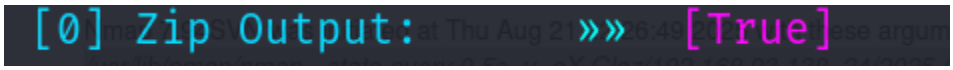
To make scan results easily accessible, Glaz provides a search functionality for previous scans and it gives the user an option to scan for previous files with quick scan or deep scan and the way it does it is by searching the base64 string inside the hidden DATA file.
each time Glaz completes a scan it creates a hidden “DATA” file and this file contains a base64 string that encodes to “This is a glaz mark” and another line that tells the user to not delete this file in case he finds it and views it as this file is critical for the Glaz tool and without it Glaz wouldn't be able to find the scan in case the user decides to view it within the Glaz framework.
This function allows the user to initiate a search across the filesystem to locate any past reports generated by the tool. The findings are then presented in a clean, date-sorted list, allowing the user to select any specific scan and dive into its detailed results, such as viewing the vulnerability report, checking brute-force logs, or opening the full HTML report.

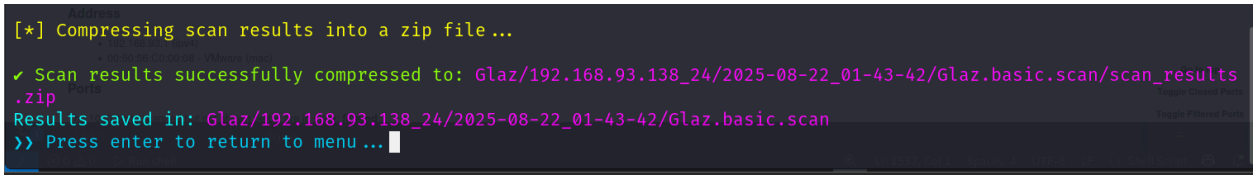




4.4 Allow to save all results into a Zip file.

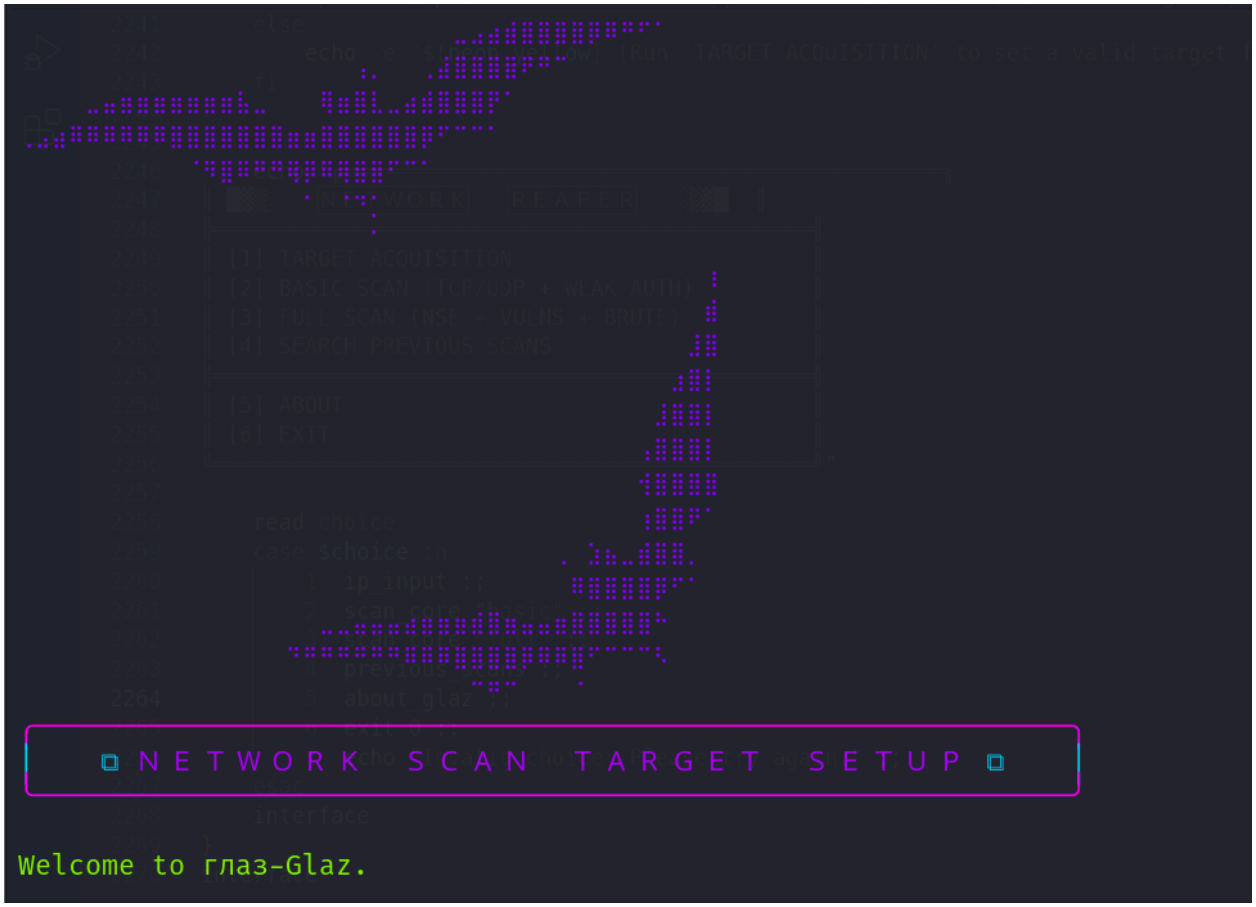
To simplify the management of scan results, Glaz provides a built-in feature to save all output into a compressed Zip file. This is an optional feature that can be enabled from the scan configuration menu before the scan begins. When all the stages are finished, the script gathers all generated artifacts, such as the detailed Nmap logs and the web-based HTML report, and packages them together into a single, portable file, making it straightforward to archive the results.



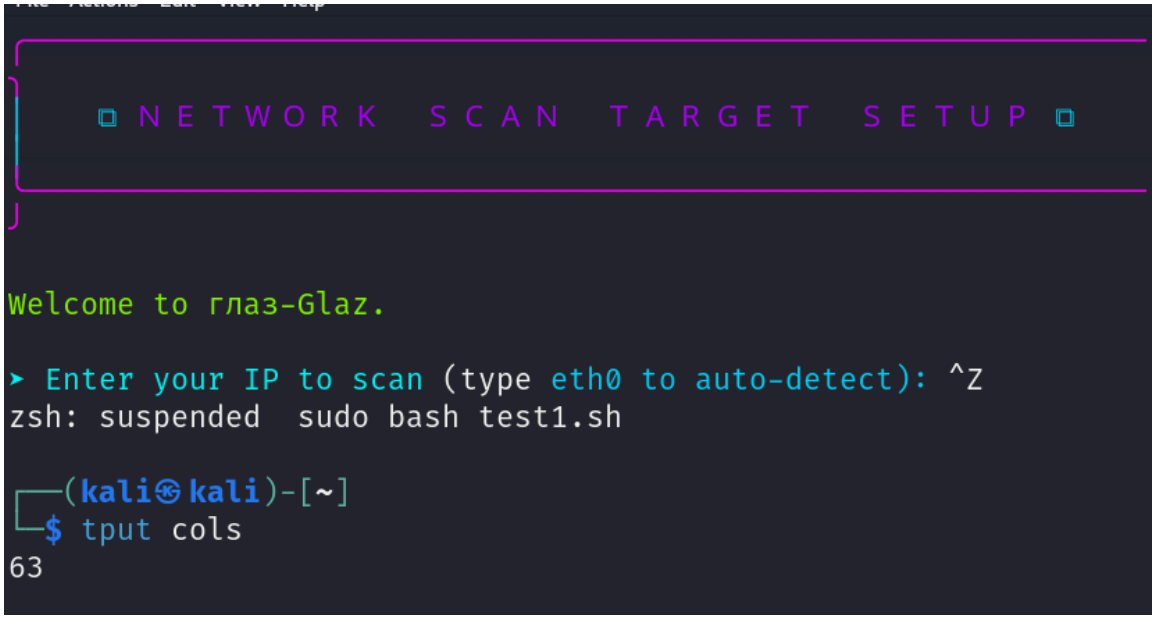


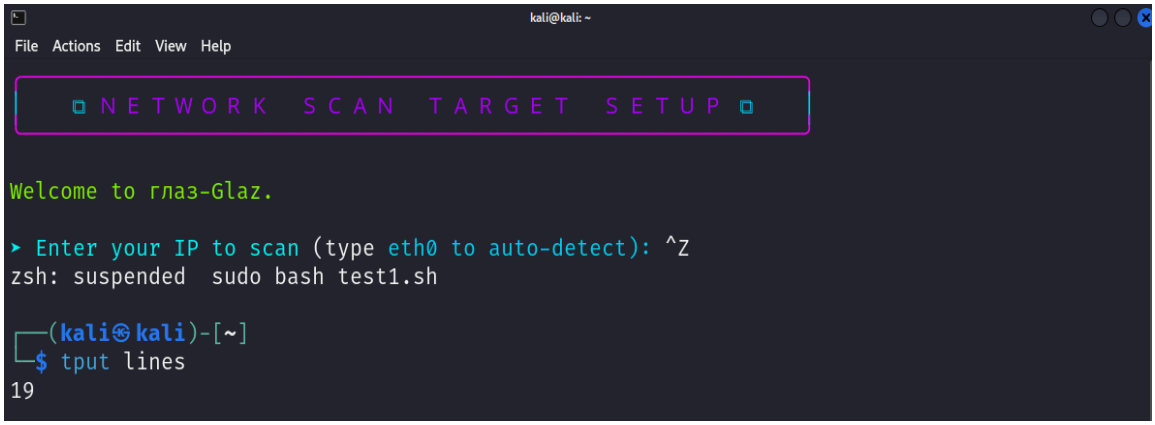
5. Creativity and User Interface

Glaz is designed with a strong, thematic aesthetic and user interface. The **ascii_art** functions are used to display stylized banners that each time a different banner displays in a random order and also random color , and a neon color theme is used throughout the script to create an engaging and visually distinct "cyberpunk" aesthetic. This improves the user experience by making the tool more memorable, readable and pleasant to the eyes.



And also I set the ascii arts to only appear if the terminal width is above 70 and height above 20 Using tput cols and tput lines
If its any less, then the ascii art wouldn't show up on the terminal to not mess it.





Furthemore i made the nmap output to show up only in one singular line and to change the output in live time without making any new lines to keep the terminal clean and aesthetic

The script contains many more cool designs and features that add up to make the script really pleasant, clean for the user and stand out.